BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Hồ Hoàng Khang
MSSV / Lớp	3123410150

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

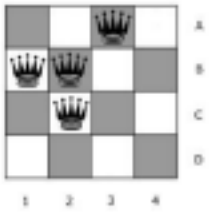
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

A3, B1, C2, D4

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☑ B) Cực trị cục bộ (local optimum)

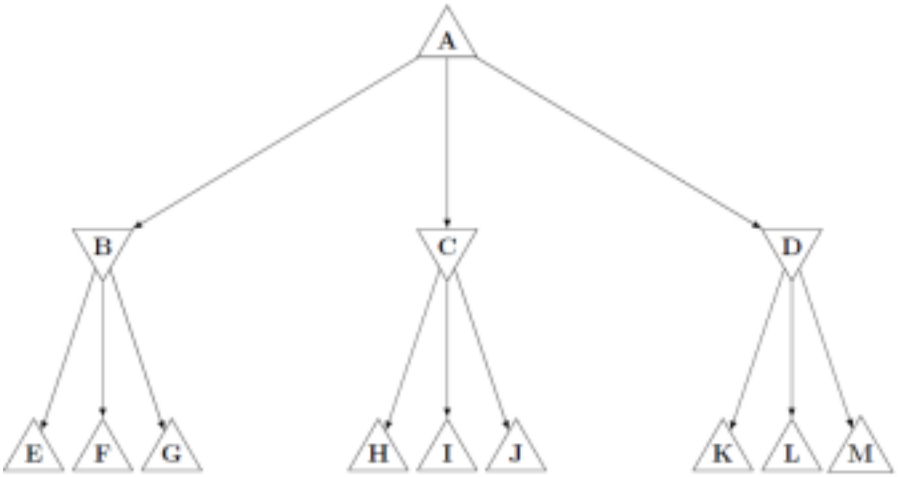
☐ C) Cực trị toàn cục (global optimum)

☑ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Các nút không được thăm là I, J, F, G.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

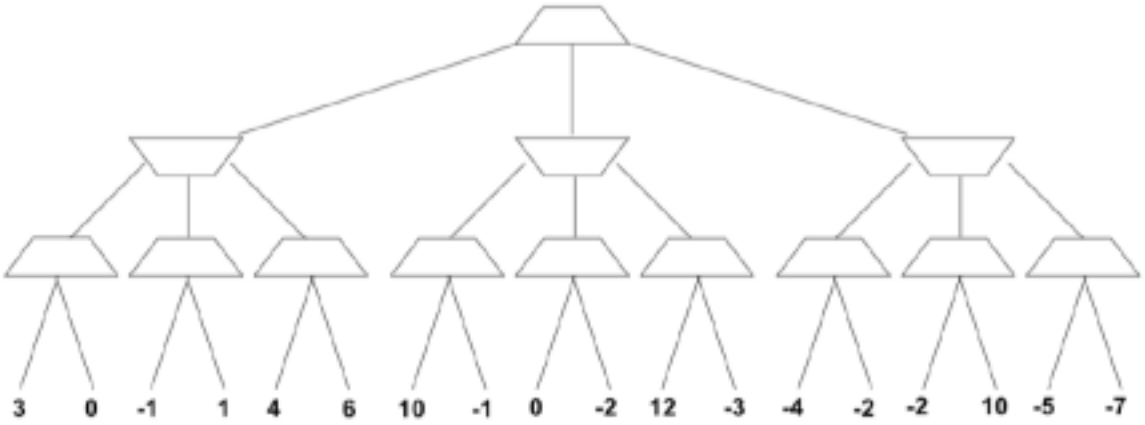
☐ Có

☑ Không

Giải thích: $f(s)$ ở độ sâu nông chỉ là giá trị ước lượng, có thể sắp xếp sai thứ tự tối ưu thực sự và vô tình cắt tỉa mất nhánh tốt nhất ở độ sâu hơn

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

1. Giá trị α, β và giá trị trả về tại mỗi nút trong quá trình duyệt

- **Nhánh B (MIN):**
  - **Nút E (MAX):** $\max(3, 0) = 3$
  - **Nút F (MAX):** $\max(-1, 1) = 1$
  - **Nút G (MAX):** $\max(4, 6) = 6$
  - **Nút B (MIN):** $\alpha = -\infty, \beta = 1$. Giá trị trả về = $\min(3, 1, 6) = 1$
  - **Nút A (MAX):** Cập nhật $\alpha = \max(-\infty, 1) = 1, \beta = +\infty$
- **Nhánh C (MIN):**
  - **Nút H (MAX):** $\max(10, -1) = 10$
  - **Nút I (MAX):** $\max(0, -2) = 0$
  - **Nút C (MIN):** $\alpha = 1, \beta = 0$. Giá trị trả về = $\min(10, 0) = 0$
  - **Nút A (MAX):** Giữ nguyên $\alpha = 1, \beta = +\infty$
- **Nhánh D (MIN):**
  - **Nút J (MAX):** $\max(12, -3) = 12$
  - **Nút K (MAX):** $\max(-4, -2) = -2$
  - **Nút D (MIN):** Sau khi tính xong K, $\beta = -2$. Do $\beta(-2) \leq \alpha(1)$ nên xảy ra **cắt tỉa α tại D**.
  - Giá trị tại nút D: $\alpha = 1, \beta = -2$. Giá trị trả về = -2

2. Đánh dấu các nhánh bị cắt tỉa

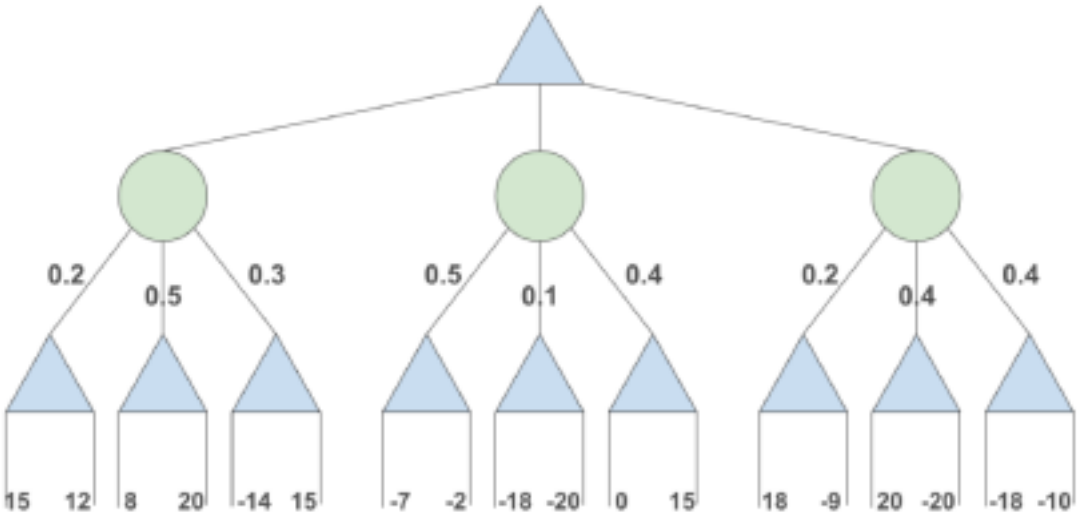
- **Đánh dấu X** trên cạnh nối từ **nút** D **đi xuống nút** L (cắt bỏ hoàn toàn nút L cùng các lá bên dưới).

3. Giá trị Minimax tại gốc và Nước đi tối ưu của MAX

- **Giá trị Minimax tại nút gốc** A**:** 1 (do $\max(1, 0, -2) = 1$)
- **Nước đi tối ưu của MAX:** Chọn **nhánh** B (đi sang cây con bên trái)

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

1. Giá trị tại các nút MAX tầng dưới (chọn max 2 lá):

- Cây C1:
  - Nút MAX 1: max(15, 12) = 15
  - Nút MAX 2: max(8, 20) = 20
  - Nút MAX 3: max(-14, 15) = 15
- Cây C2:
  - Nút MAX 4: max(-7, -2) = -2
  - Nút MAX 5: max(-18, -20) = -18
  - Nút MAX 6: max(0, 15) = 15
- Cây C3:
  - Nút MAX 7: max(18, -9) = 18
  - Nút MAX 8: max(20, -20) = 20
  - Nút MAX 9: max(-18, -10) = -10

2. Giá trị tại 3 Chance Node (tính trung bình có trọng số):

- Chance Node 1: (0.2 * 15) + (0.5 * 20) + (0.3 * 15) = 3 + 10 + 4.5 = 17.5
- Chance Node 2: (0.5 * -2) + (0.1 * -18) + (0.4 * 15) = -1 - 1.8 + 6 = 3.2
- Chance Node 3: (0.2 * 18) + (0.4 * 20) + (0.4 * -10) = 3.6 + 8 - 4 = 7.6

3. Giá trị tại Nút Gốc MAX:

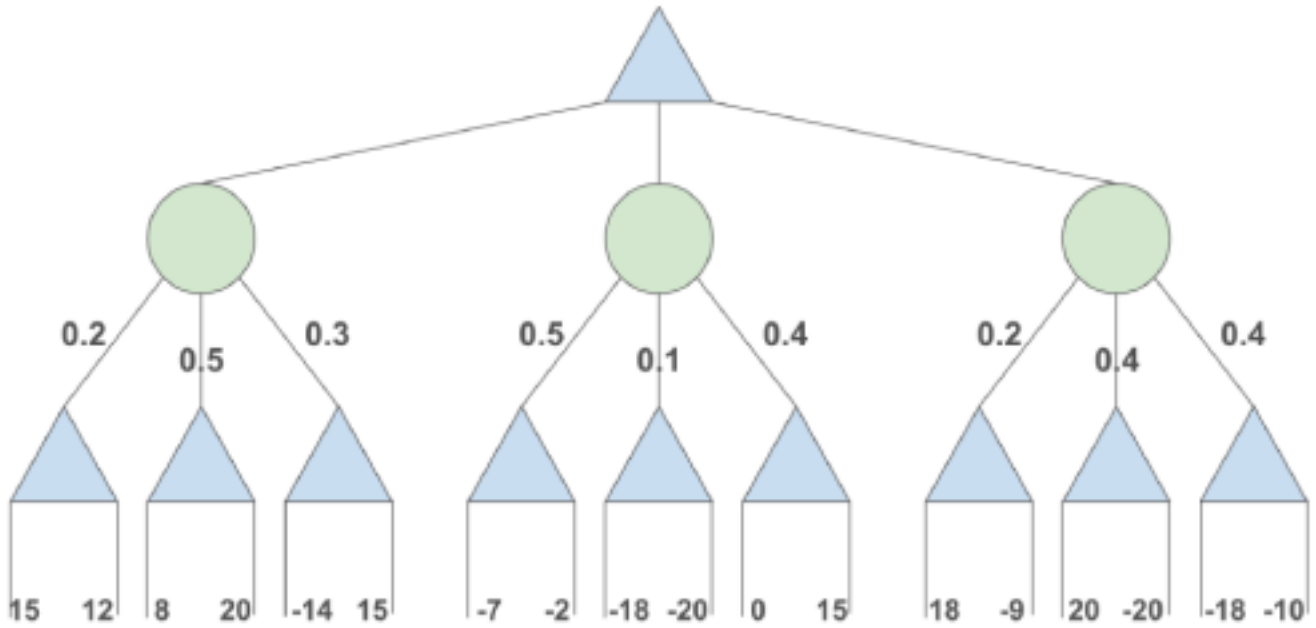
- V_gốc = max(17.5, 3.2, 7.6) = 17.5

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có nút nào có thể bị cắt tỉa mà vẫn đảm bảo kết quả chính xác.

Giải thích: Trong cây Expectimax, các chance node tính tổng trung bình có trọng số từ các nút con. Khi chưa biết giới hạn của các nút lá, một lá chưa thăm có thể mang giá trị cực kỳ lớn (+infinity), làm thay đổi hoàn toàn giá trị kỳ vọng của chance node đó và thay đổi luôn quyết định của nút gốc MAX.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Nút bị cắt tỉa: Nút MAX 6 (chứa 2 lá 0 và 15) thuộc Chance Node 2 bị cắt tỉa.

Giải thích:

- Khi duyệt xong Chance Node 1, nút gốc MAX có giá trị sàn hiện tại $\alpha = 17.5$.
- Giới hạn trên của các lá là 20.
- Tại Chance Node 2:
  - Nút MAX 4 = -2 → Đóng góp: $0.5 \times (-2) = -1$
  - Nút MAX 5 = -18 → Đóng góp: $0.1 \times (-18) = -1.8$
  - Giá trị tối đa mà Chance Node 2 có thể đạt được (khi nút MAX 6 đạt giá trị cực đại 20) là: $(-1) + (-1.8) + (0.4 \times 20) = 5.2$
  - Vì $5.2 < \alpha(17.5)$, Chance Node 2 không bao giờ vượt qua Chance Node 1.
  - Do đó, cắt tỉa ngay nút MAX 6 (không cần duyệt 2 lá $0, 15$).
- Tại Chance Node 3: Giá trị tối đa có thể đạt $20 > 17.5$ nên bắt buộc phải duyệt toàn bộ, không có nhánh nào bị cắt.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Sự khác nhau giữa Forward Checking và AC-3:

1. **Phạm vi lan truyền ràng buộc:**
- **Forward Checking:** Chỉ lan truyền ràng buộc trực tiếp từ biến vừa được gán giá trị đến các biến kề (chưa gán) kết nối trực tiếp với nó.
- **AC-3:** Lan truyền ràng buộc trên toàn bộ đồ thị mạng ràng buộc (kể cả giữa các biến chưa gán với nhau) cho đến khi đạt trạng thái cung nhất quán (Arc Consistency) hoặc phát hiện miền giá trị rỗng.
2. **Thời điểm thực hiện:**
- **Forward Checking:** Được thực hiện **trong quá trình tìm kiếm (backtracking search)** ngay sau mỗi bước gán giá trị cho một biến.
- **AC-3:** Có thể thực hiện **trước khi tìm kiếm** (như một bước tiền xử lý) hoặc **kết hợp trong khi tìm kiếm** (thuật toán MAC - Maintaining Arc Consistency).
3. **Chi phí tính toán:**
- **Forward Checking:** Chi phí tính toán **thấp** (độ phức tạp $O(d)$ cho mỗi biến kề, với d là kích thước miền giá trị).
- **AC-3:** Chi phí tính toán **cao hơn nhiều** (độ phức tạp thời gian trường hợp xấu nhất là $O(r \cdot d^3)$ hoặc $O(r \cdot d^2)$ với r là số lượng ràng buộc/cung).

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

Cách hoạt động: Ưu tiên chọn biến chưa được gán giá trị mà có **số lượng giá trị hợp lệ còn lại trong miền giá trị là ít nhất** (biến bị ràng buộc nhiều nhất / hầu như không còn lựa chọn).

Lý do sử dụng (Nguyên lý "Fail-first"):

- Giúp phát hiện thất bại (bế tắc) càng sớm càng tốt trong cây tìm kiếm.
- Nếu chọn một biến chỉ còn rất ít lựa chọn, nếu nó dẫn đến bế tắc thì thuật toán sẽ phát hiện ngay và quay lui (backtrack) sớm, tránh lãng phí thời gian duyệt sâu vào các nhánh vô nghiệm.
- Giúp thu hẹp tối đa hệ số phân nhánh (branching factor) ở các tầng trên của cây tìm kiếm.

ii) Least Constraining Value (LCV)

Cách hoạt động: Ưu tiên chọn **giá trị trong miền giá trị** sao cho giá trị đó **ít làm giảm bớt số lựa chọn nhất** cho các biến kề chưa gán.

Lý do sử dụng (Nguyên lý "Fail-last"):

- Trong bước chọn *giá trị* cho một biến (khác với bước chọn *biến* của MRV), mục tiêu là tìm ra nghiệm càng nhanh càng tốt mà không làm cạn kiệt không gian nghiệm của các biến khác.
- Giúp giữ tối đa khả năng mở rộng nghiệm cho các bước tiếp theo, từ đó giảm thiểu nguy cơ xảy ra xung đột và phải quay lui (backtrack).

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

1. Tập biến (Variables - X):

Mỗi máy bay $i \in \{A, B, C, D, E\}$ được biểu diễn bởi 2 biến:

- T_i: Khung giờ của máy bay i.
- R_i: Đường băng của máy bay i.

Tổng cộng có 10 biến: $\{T_A, T_B, T_C, T_D, T_E\}$ và $\{R_A, R_B, R_C, R_D, R_E\}$.

2. Miền giá trị (Domains - D):

- Miền giá trị cho các biến khung giờ: $D_{T_i} = \{1, 2, 3, 4\}$ với mọi $i \in \{A, B, C, D, E\}$.
- Miền giá trị cho các biến đường băng: $D_{R_i} = \{Quốctế, Nộiđịa\}$ với mọi $i \in \{A, B, C, D, E\}$.

3. Các ràng buộc (Constraints - C):

- **Ràng buộc đơn/nguyên (Unary Constraints):**
  - B bị hỏng 1 động cơ, hạ cánh khung giờ 1:
    - $T_B = 1$
  - D chỉ đến hạ cánh từ khung giờ 3 trở đi:
    - $T_D \geq 3 (hoặc T_D \in \{3, 4\})$
  - A chỉ có thể chờ tối đa đến khung giờ 2:
    - $T_A \leq 2 (hoặc T_A \in \{1, 2\})$
- **Ràng buộc nhị phân (Binary Constraints):**
  - D phải hạ cánh trước khi C cất cánh:
    - $T_D < T_C$
  - Không có hai máy bay nào trùng khung giờ trên cùng một đường băng (với mọi cặp máy bay $i \neq j$):

$$T_i = T_j \Rightarrow (R_i \neq R_J) \; hoặc \; \neg(T_i = T_J \wedge R_i = R_J)$$

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

1. Xác định miền giá trị mới của các biến khung giờ (T_i):

Do đường băng của từng máy bay đã được cố định:

- **Đường băng Quốc tế:** Máy bay A, B, C
- **Đường băng Nội địa:** Máy bay D, E

Nên bài toán chỉ còn lại 5 biến khung giờ $\{T_A, T_B, T_C, T_D, T_E\}$ với miền giá trị ban đầu là $\{1, 2, 3, 4\}$.

Áp dụng các **ràng buộc đơn (unary constraints)** để thu hẹp miền giá trị:

- $T_B = 1 \Rightarrow D_{T_B} = \{1\}$
- $T_A \leq 2 \Rightarrow D_{T_A} = \{1, 2\}$
- $T_D \geq 3 \Rightarrow D_{T_D} = \{3, 4\}$
- Ràng buộc $T_D < T_C$: Vì $T_D \geq 3$ nên T_C phải lớn hơn $3 \Rightarrow D_{T_C} = \{4\}$
- T_E chưa bị ràng buộc đơn nào $\Rightarrow D_{T_E} = \{1, 2, 3, 4\}$

Tóm tắt miền giá trị mới:

- $D_{T_A} = \{1, 2\}$
- $D_{T_B} = \{1\}$
- $D_{T_C} = \{4\}$
- $D_{T_D} = \{3, 4\}$
- $D_{T_E} = \{1, 2, 3, 4\}$

2. Các ràng buộc nhị phân (Binary Constraints) còn lại:

- **Cùng nhóm Quốc tế (không được trùng khung giờ):**
  - $T_A \neq T_B$
  - $T_B \neq T_C$
  - $T_A \neq T_C$
- **Cùng nhóm Nội địa (không được trùng khung giờ):**
  - $T_D \neq T_E$
- **Ràng buộc nối chuyến (liên nhóm):**
  - $T_D < T_C$

3. Đồ thị ràng buộc (Constraint Graph):

Các biến là các đỉnh, các cạnh nối giữa hai biến biểu diễn ràng buộc nhị phân giữa chúng:

- **Tập đỉnh (Nodes):** $\{T_A, T_B, T_C, T_D, T_E\}$
- **Tập cạnh (Edges):**
  - Cạnh (T_A, T_B) với quan hệ $T_A \neq T_B$
  - Cạnh (T_B, T_C) với quan hệ $T_B \neq T_C$
  - Cạnh (T_A, T_C) với quan hệ $T_A \neq T_C$
  - Cạnh (T_D, T_E) với quan hệ $T_D \neq T_E$

- Cạnh (T_D, T_C) với quan hệ $T_D < T_C$

Mô tả hình vẽ đồ thị để bạn tự phác thảo vào bài:

[T_A] -------- (≠) -------- [T_B]

| **|**

(≠) **(≠)**

| **|**

[T_C] <------- (<) -------- [T_D] -------- (≠) -------- [T_E]

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

1. Khởi tạo miền giá trị từ các ràng buộc đơn (Unary):

- $D_A = \{1, 2\}$ (do $A \leq 2$)
- $D_B = \{1\}$ (do $B = 1$)
- $D_C = \{1, 2, 3, 4\}$
- $D_D = \{3, 4\}$ (do $D \geq 3$)
- $D_E = \{1, 2, 3, 4\}$

2. Gạch bỏ các giá trị bị loại do vi phạm cung nhất quán:

- **Do** $A \neq B$ **(**$B = 1$**):** Loại 1 khỏi $D_A \implies D_A = \{\cancel{1}, 2\} = \{2\}$
- **Do** $D < C$ **(**$D \geq 3$**):** C phải $> 3 \implies$ Loại 1, 2, 3 khỏi $D_C \implies D_C = \{\cancel{1}, \cancel{2}, \cancel{3}, 4\} = \{4\}$
- **Do** $D < C$ **(**$C = 4$**):** D phải $< 4 \implies$ Loại 4 khỏi $D_D \implies D_D = \{3, \cancel{4}\} = \{3\}$
- **Do** $E \neq D$ **(**$D = 3$**):** Loại 3 khỏi $D_E \implies D_E = \{1, 2, \cancel{3}, 4\} = \{1, 2, 4\}$

Kết quả miền giá trị còn lại:

- $D_A = \{2\}$
- $D_B = \{1\}$
- $D_C = \{4\}$
- $D_D = \{3\}$

- $D_E = \{1, 2, 4\}$

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

1. Áp dụng ràng buộc Unary trước khi duyệt: Miền ban đầu: $D_A = \{1, 2\}$, $D_B = \{1\}$, $D_C = \{1, 2, 3, 4\}$, $D_D = \{3, 4\}$, $D_E = \{1, 2, 3, 4\}$.

2. Thứ tự phát sinh các phép gán:

1. $(B, 1)$: Chọn B (miền nhỏ nhất = 1). FC lọc $D_A = \{2\}$, $D_C = \{2, 3, 4\}$.
2. $(A, 2)$: Chọn A (miền nhỏ nhất = 1). FC lọc $D_C = \{3, 4\}$.
3. $(C, 3)$: Chọn C (hòa với D miền = 2, chọn ký tự C trước). FC lọc $D < 3 \implies D_D = \emptyset$ (**Dead end / Bế tắc**).
   - → **Hoàn tác phép gán** $(C, 3)$.
4. $(C, 4)$: Thử giá trị tiếp theo của C. FC lọc $D < 4 \implies D_D = \{3\}$.
5. $(D, 3)$: Chọn D (miền nhỏ nhất = 1). FC lọc $E \neq 3 \implies D_E = \{1, 2, 4\}$.
6. $(E, 1)$: Chọn E, lấy giá trị nhỏ nhất còn lại là 1.

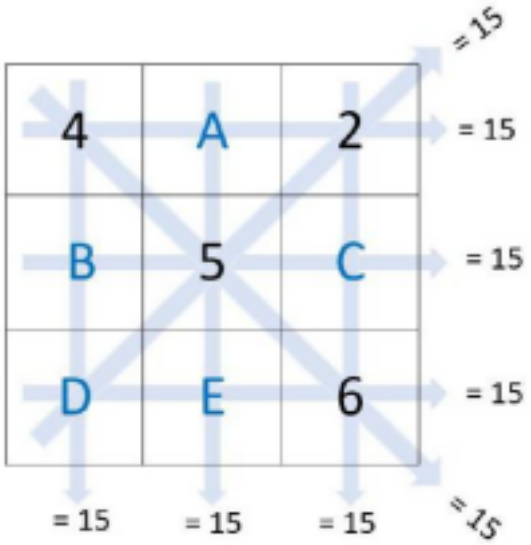
Danh sách các cặp (biến, giá trị) theo đúng thứ tự phát sinh để điền vào bài:

$$(B, 1) \rightarrow (A, 2) \rightarrow (C, 3)[hoàntác] \rightarrow (C, 4) \rightarrow (D, 3) \rightarrow (E, 1)$$

Nghiệm thu được: $A = 2, B = 1, C = 4, D = 3, E = 1$.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

1. Các biến (Variables - X):

Bỏ qua các ô đã biết $(4, 2, 5, 6)$, chỉ tạo biến cho 5 ô chưa biết:

$$X = \{A, B, C, D, E\}$$

2. Miền giá trị (Domains - D): Các số đã dùng là $\{2, 4, 5, 6\}$. Miền giá trị của 5 biến là tập các số chưa dùng từ 1 đến 9:

$$D_A = D_B = D_C = D_D = D_E = \{1, 3, 7, 8, 9\}$$

3. Các ràng buộc (Constraints - C):

- **Ràng buộc khác nhau:** $AllDifferent(A, B, C, D, E)$
- **Ràng buộc tổng hàng/cột/đường chéo bằng 15:**
  - **Hàng 1:** $A + 4 + 2 = 15 \implies A = 9$
  - **Hàng 2:** $B + 5 + C = 15 \implies B + C = 10$
  - **Hàng 3:** $D + E + 6 = 15 \implies D + E = 9$
  - **Cột 1:** $B + D + 4 = 15 \implies B + D = 11$
  - **Cột 2:** $A + 5 + E = 15 \implies A + E = 10$
  - **Cột 3:** $C + 2 + 6 = 15 \implies C = 7$
  - **Đường chéo phụ:** $D + 5 + 2 = 15 \implies D = 3$

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

1. Các ràng buộc nhị phân thu được từ phương trình tổng:

- $B + C = 10 \implies$ Cạnh giữa B và C
- $D + E = 9 \implies$ Cạnh giữa D và E
- $B + D = 11 \implies$ Cạnh giữa B và D

- $A + E = 10 \implies$ Cạnh giữa A và E

(Lưu ý: Các phương trình $A = 9$, $C = 7$, $D = 3$ là ràng buộc đơn/unary, chỉ thu hẹp miền giá trị của biến nên không vẽ cạnh nhị phân).

2. Mô tả đồ thị ràng buộc nhị phân:

- **Tập đỉnh (Nodes):** $\{A, B, C, D, E\}$
- **Tập cạnh (Edges):** $(A, E), (B, C), (B, D), (D, E)$

Sơ đồ đồ thị:

[A] ---------- [E]

|

|

[C] -- [B] ----- [D]

(Biến A nối với E; C nối với B; B nối với D; D nối với E tạo thành đường đi liên kết giữa các biến).

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

1. Áp dụng AC-3 (Arc Consistency) để thu hẹp miền giá trị:

- **Miền ban đầu:** $D_A = D_B = D_C = D_D = D_E = \{1, 3, 7, 8, 9\}$
- **Rút gọn miền từ các ràng buộc đơn (Unary):**
  - Hàng 1: $A + 4 + 2 = 15 \implies A = 9 \implies \mathbf{D_A = \{9\}}$
  - Cột 3: $C + 2 + 6 = 15 \implies C = 7 \implies \mathbf{D_C = \{7\}}$
  - Đường chéo phụ: $D + 5 + 2 = 15 \implies D = 3 \implies \mathbf{D_D = \{3\}}$
- **Lan truyền cung nhất quán (AC-3) sang các biến kề:**
  - **Cung** (E, A) **với** $A + E = 10$**:** Vì $A = 9 \implies E = 10 - 9 = 1 \implies \mathbf{D_E = \{1\}}$
  - **Cung** (B, C) **với** $B + C = 10$ **(hoặc** (B, D) **với** $B + D = 11$**):**
    - Từ $C = 7 \implies B = 10 - 7 = 3$ (hoặc từ $D = 3 \implies B = 11 - 3 = 8$).
    - Kiểm tra với cột 1 ($4 + B + D = 15$): $B = 8$ thỏa mãn đồng thời các ràng buộc $\implies \mathbf{D_B = \{8\}}$.

2. Nghiệm tìm được cho các ô chữ cái:

- $A = 9$
- $B = 8$
- $C = 7$
- $D = 3$
- $E = 1$

Ma trận Magic Square 3×3 hoàn chỉnh:

| 4 | 9 | 2 |

| 8 | 5 | 7 |

| 3 | 1 | 6 |

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

1. Khởi tạo:

- Các biến và miền giá trị ban đầu:
  - $D_{AJ} = \{\text{Đỏ}, Vàng, Xanhlá\}$ (kích thước = 3)
  - $D_{Sunny} = \{\text{Đỏ}, Vàng, Hồng\}$ (kích thước = 3)
  - $D_{Ellyse} = \{\text{Đỏ}, Hồng\}$ (kích thước = 2)
- Thứ tự ưu tiên phá hòa theo cầu vồng: **Đỏ** → **Vàng** → **Xanh lá** → **Hồng**

2. Diễn biến thuật toán gán giá trị:

- **Bước 1: Chọn biến theo MRV**
  - Kích thước miền: $Ellyse(2) < AJ(3) = Sunny(3)$.
  - Chọn biến **Ellyse**.
  - **Chọn giá trị theo LCV (kiểm tra mức độ ảnh hưởng đến biến kề AJ và Sunny):**
    - Thử giá trị **Đỏ**: Loại "Đỏ" khỏi AJ và Sunny $\Longrightarrow$ làm giảm 1 giá trị của AJ và 1 giá trị của Sunny (tổng giảm = 2).
    - Thử giá trị **Hồng**: Loại "Hồng" khỏi Sunny $\Longrightarrow$ làm giảm 1 giá trị của Sunny (tổng giảm = 1).
    - Giá trị **Hồng** làm giảm ít lựa chọn hơn $\Longrightarrow$ Chọn **Hồng**.
  - **Gán:** $Ellyse = Hồng$
    - Miền giá trị sau khi lọc:
      - $D_{AJ} = \{\text{Đỏ}, Vàng, Xanhlá\}$ (kích thước = 3)
      - $D_{Sunny} = \{\text{Đỏ}, Vàng\}$ (kích thước = 2)
- **Bước 2: Chọn biến tiếp theo theo MRV**
  - Kích thước miền: $Sunny(2) < AJ(3)$.
  - Chọn biến **Sunny**.
  - **Chọn giá trị theo LCV (ảnh hưởng đến AJ):**
    - Thử **Đỏ**: Loại "Đỏ" khỏi AJ $\Longrightarrow$ làm giảm 1 lựa chọn của AJ.
    - Thử **Vàng**: Loại "Vàng" khỏi AJ $\Longrightarrow$ làm giảm 1 lựa chọn của AJ.
    - Đều làm giảm 1 lựa chọn (hòa LCV) $\Longrightarrow$ Phá hòa theo thứ tự cầu vồng (Đỏ trước Vàng).
    - Chọn **Đỏ**.

- **Gán:** $Sunny = \text{Đỏ}$
  - Miền giá trị sau khi lọc:
    - $D_{AJ} = \{Vàng, Xanhlá\}$ (kích thước = 2)

- **Bước 3: Chọn biến cuối cùng**
  - Chọn biến **AJ**.
  - Ưu tiên chọn theo thứ tự cầu vồng: **Vàng** (trước Xanh lá).
  - **Gán:** $AJ = Vàng$

3. KẾT QUẢ:

- **Số lần backtrack: 0 lần** (Tìm được nghiệm ngay trong lần gán đầu tiên, không gặp bế tắc).
- **Thứ tự các phép gán:**
  1. $Ellyse = Hồng$
  2. $Sunny = \text{Đỏ}$
  3. $AJ = Vàng$

(Nghiệm hoàn chỉnh: Ellyse = Hồng, Sunny = Đỏ, AJ = Vàng)

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

1. Khởi tạo:

- Các biến và miền giá trị ban đầu:
  - $D_{AJ} = \{\text{Đỏ}, Vàng, Xanhlá\}$ (kích thước = 3)
  - $D_{Sunny} = \{\text{Đỏ}, Vàng, Hồng\}$ (kích thước = 3)
  - $D_{Ellyse} = \{\text{Đỏ}, Hồng\}$ (kích thước = 2)
- Phá hòa theo thứ tự cầu vồng: **Đỏ** → **Vàng** → **Xanh lá** → **Hồng**

2. Diễn biến thuật toán gán giá trị:

- **Bước 1: Chọn biến theo Maximum Remaining Values (biến có miền giá trị LỚN NHẤT)**
  - Kích thước miền: $AJ(3) = Sunny(3) > Ellyse(2)$.
  - Hòa kích thước giữa AJ và Sunny $\Longrightarrow$ Phá hòa chọn theo thứ tự tên/cầu vồng: Chọn **AJ**.
  - **Chọn giá trị theo Most Constraining Value (giá trị LÀM GIẢM NHIỀU LỰA CHỌN NHẤT của các biến kề):**
    - Thử **Đỏ**: Loại "Đỏ" khỏi Sunny và Ellyse $\Longrightarrow$ Giảm 2 lựa chọn (ảnh hưởng lớn nhất).
    - Thử **Vàng**: Loại "Vàng" khỏi Sunny $\Longrightarrow$ Giảm 1 lựa chọn.
    - Thử **Xanh lá**: Không trùng với biến nào $\Longrightarrow$ Giảm 0 lựa chọn.
    - $\Longrightarrow$ Chọn giá trị **Đỏ**.
  - **Gán:** $AJ = \text{Đỏ}$
    - Miền giá trị còn lại:
      - $D_{Sunny} = \{Vàng, Hồng\}$ (kích thước = 2)
      - $D_{Ellyse} = \{Hồng\}$ (kích thước = 1)
- **Bước 2: Chọn biến tiếp theo theo Maximum Remaining Values**
  - Kích thước miền: $Sunny(2) > Ellyse(1)$.
  - Chọn biến **Sunny**.
  - **Chọn giá trị theo Most Constraining Value:**
    - Thử **Hồng**: Loại "Hồng" khỏi Ellyse $\Longrightarrow$ Giảm 1 lựa chọn của Ellyse (ảnh hưởng lớn nhất làm $D_{\text{Ellyse}} \to \emptyset$).

        - Thử **Vàng**: Không loại giá trị nào của Ellyse $\Longrightarrow$ Giảm 0 lựa chọn.
        - $\Longrightarrow$ Chọn giá trị **Hồng**.
    - **Gán:** $Sunny = Hồng$
        - Miền giá trị còn lại:
            - $D_{\text{Ellyse}} = \emptyset$ (**Dead end / Bế tắc!**)
- **Bước 3: Backtrack (Quay lui)**
    - **Hoàn tác** $Sunny = Hồng$ (Backtrack 1 lần).
    - Thử giá trị tiếp theo cho Sunny: **Vàng**.
    - **Gán:** $Sunny = Vàng$
        - Miền giá trị còn lại:
            - $D_{Ellyse} = \{Hồng\}$ (kích thước = 1)
- **Bước 4: Chọn biến cuối cùng**
    - Chọn biến **Ellyse**.
    - **Gán:** $Ellyse = Hồng$
        - Tìm ra nghiệm thành công!

3. KẾT QUẢ & SO SÁNH:

- **Số lần backtrack: 1 lần** (Gán $Sunny = Hồng$ dẫn đến bế tắc $D_{\text{Ellyse}} = \emptyset$ nên phải quay lui).
- **Thứ tự các phép gán:**
    - $AJ = Đỏ$
    - $Sunny = Hồng$ *(Dead End → Backtrack)*
    - $Sunny = Vàng$
    - $Ellyse = Hồng$
- **Giải thích vì sao số lần backtrack nhiều hơn câu (a):**
    - **Maximum Remaining Values** đi ngược lại với triết lý *"Fail-first"* của MRV: Thuật toán trì hoãn việc gán cho biến bị ràng buộc nhiều nhất (Ellyse), làm tăng khả năng đi sâu vào các nhánh vô nghiệm.
    - **Most Constraining Value** đi ngược lại với triết lý *"Fail-last"* của LCV: Việc cố tình chọn giá trị triệt hạ nhiều lựa chọn của các biến chưa gán làm miền giá trị của biến kề bị cạn kiệt sớm, trực tiếp dẫn đến xung đột và buộc thuật toán phải backtrack.

--- Hết ---